PLAN DE TRABAJO – PROYECTO

CENTRO EDUCATIVO

"Educar para Transformar" – Aplicación Móvil

Metodología de Sistemas II – TUP 2026

Trabajo Práctico Integrador – Parte Aplicación Móvil

NOMBRE DEL EQUIPO DE TRABAJO

[Completar nombre del equipo]

APELLIDO Y NOMBRE DE LOS INTEGRANTES

Integrante 1 – Trangoni, Fabián

Integrante 2 – Mazza, Bruno

1. DESCRIPCIÓN DEL PROYECTO

1.1 Enunciado del problema

El Centro Educativo “Educar para Transformar” iniciará sus operaciones en marzo de 2027 y contará con un Sistema de Gestión web que centraliza la información académica, deportiva y de servicios (transporte y comedor). Sin embargo, padres, estudiantes y docentes necesitan consultar información y realizar gestiones frecuentes (inscribir a un hijo en un deporte, revisar el recorrido de transporte, ver los cursos asignados) en cualquier momento y desde su celular, sin depender de una computadora ni de un navegador.

El problema central es la ausencia de un canal móvil que acerque el sistema a sus usuarios, que permita avisarles de inmediato los eventos relevantes (notificaciones push) y que mantenga la misma integridad de datos, reglas de negocio y control de acceso por rol ya definidos en el sistema web, protegiendo además los datos personales de menores en un dispositivo que puede perderse o compartirse.

Alcance y supuestos

- La app es un **nuevo cliente del backend existente**: no duplica reglas de negocio y consume la API REST prevista en RNF-06 y RNF-09 del plan web.
- Perfiles móviles: **Estudiante, Padre y Docente**, más una sección pública. Administrador y Directivo continúan operando desde la web.
- Plataformas: **Android e iOS** desde una única base de código.
- Se asume la API del plan web desplegada y estable al iniciar este plan (Sprints 1 a 5 del plan web completados).
- Fuera de alcance (permanecen en la web): ABM de alumnos y profesores, configuración académica, gestión de usuarios y permisos, reportes, formulario de inscripción institucional y ofertas de empleo (RF-05, RF-06, RF-07, RF-11, RF-13, RF-14, RF-16).

1.2 Requerimientos

Los requerimientos móviles derivan de los del plan web (columna “Origen”) y se completan con los propios del canal móvil. Se clasifican en Funcionales (RFM) y No Funcionales (RNFM).

Requerimientos Funcionales (RFM)

Código	Requerimiento funcional	Módulo	Origen (plan web)
RFM-01	Permitir el inicio de sesión de padres, estudiantes y docentes, y opcionalmente el desbloqueo de la app con biometría del dispositivo.	App – Acceso	RF-04, RF-14
RFM-02	Permitir consultar, sin autenticarse, información institucional, niveles educativos, bienestar estudiantil, noticias y galería de imágenes.	App – Institucional	RF-01, RF-03
RFM-03	Permitir dejar una opinión/comentario sobre la institución.	App – Institucional	RF-02
RFM-04	Permitir al padre consultar exclusivamente la información académica de sus propios hijos.	App – Alumnos	RF-17
RFM-05	Permitir al estudiante consultar sus datos académicos, inscripciones y horarios.	App – Alumnos	RF-04
RFM-06	Permitir al padre inscribir a su hijo en hasta 2 deportes simultáneos, informando el rechazo de la API ante superposición horaria.	App – Alumnos	RF-08, RF-15

Código	Requerimiento funcional	Módulo	Origen (plan web)
RFM-07	Permitir al padre inscribir a su hijo en uno de los 4 recorridos de transporte (o modificarlo), sin duplicados.	App – Alumnos	RF-09, RF-15
RFM-08	Permitir al padre inscribir a su hijo en el servicio de comedor, sin duplicados.	App – Alumnos	RF-10, RF-15
RFM-09	Permitir al docente consultar las materias y cursos que tiene asignados.	App – Docentes	RF-12
RFM-10	Recibir notificaciones push (confirmación de inscripción, cambios de horario, novedades institucionales) según el rol del usuario.	App – Notificaciones	Nuevo
RFM-11	Permitir cerrar sesión, renovar la sesión automáticamente y solicitar un nuevo ingreso cuando el token expira.	App – Acceso	RF-04 / Nuevo
RFM-12	Mostrar la última información consultada sin conexión (indicando la fecha de sincronización) y avisar cuando no hay red.	App – Transversal	Nuevo

Requerimientos No Funcionales (RNFM)

Código	Categoría	Requerimiento no funcional
RNFM-01	Usabilidad	Interfaz que respete las guías Material Design 3 (Android) y Human Interface Guidelines (iOS), con zonas táctiles ≥ 48 dp y uso cómodo con una mano.
RNFM-02	Compatibilidad	Funcionar en Android 8.0 (API 26) o superior e iOS 15 o superior (versiones a confirmar según el framework elegido) y en pantallas de distintos tamaños.
RNFM-03	Seguridad	Comunicación solo por HTTPS; token JWT guardado en almacenamiento seguro (Keystore / Keychain); sin datos sensibles en logs ni en almacenamiento no cifrado.
RNFM-04	Seguridad / Legal	Cumplir la Ley 25.326 de Protección de Datos Personales: las notificaciones no incluyen datos sensibles del menor y la app declara el uso de datos en las tiendas.
RNFM-05	Rendimiento	Arranque de la app ≤ 3 s y consultas frecuentes ≤ 3 s sobre red 4G (coherente con RNF-05 del plan web).
RNFM-06	Disponibilidad offline	Consulta de la última información sincronizada sin conexión; las inscripciones requieren conexión.
RNFM-07	Mantenibilidad	Código en capas desacopladas por módulos, sin modificar la lógica de negocio del backend (RNF-06, RNF-08).
RNFM-08	Interoperabilidad	Consumir la API REST versionada (/api/v1) compartida con la web (RNF-09).
RNFM-09	Accesibilidad	Contraste adecuado, tamaño de fuente escalable y etiquetas de accesibilidad en los controles.
RNFM-10	Portabilidad	Una única base de código para Android e iOS.
RNFM-11	Auditoría	Las operaciones críticas realizadas desde la app se registran en el backend indicando el origen “móvil” (RNF-11).

1.3 Clasificación del tipo de Sistema de Información

La aplicación móvil **no constituye un sistema de información independiente**, sino un nuevo canal (cliente) del Sistema de Información híbrido definido en el plan web:

- **TPS:** la app captura transacciones (inscripciones a deportes, transporte y comedor) que se procesan en el mismo backend y con las mismas validaciones.
- **MIS:** no se modifica; los reportes de dirección permanecen en la web.

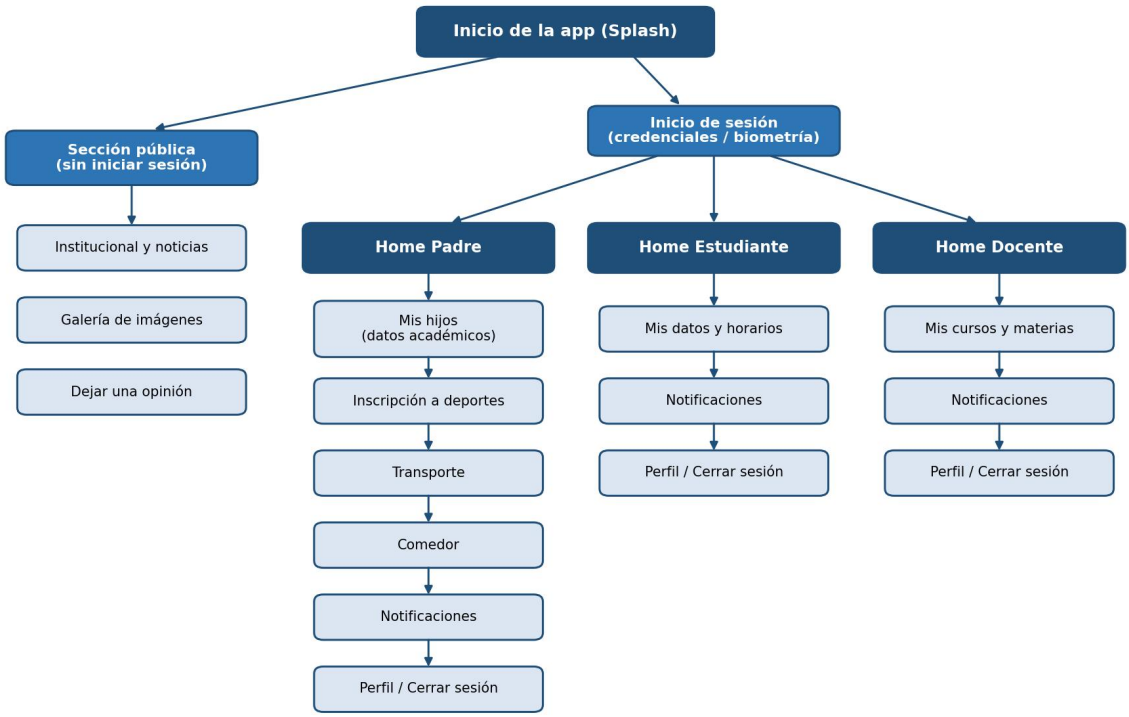
- **Portal institucional:** se extiende con acceso móvil al contenido público y al área privada por rol.

Tipo de aplicación móvil. Se compararon las alternativas de desarrollo:

Tipo	Ventajas	Desventajas	Decisión
Nativa (Kotlin / Swift)	Máximo rendimiento y acceso total al dispositivo.	Dos bases de código y más horas de desarrollo para un equipo de dos personas.	Descartada
Multiplataforma (React Native + Expo)	Una base de código para Android e iOS; coherente con React del frontend web; acceso a notificaciones, biometría y almacenamiento seguro.	Rendimiento algo menor que el nativo; dependencia del framework.	**Elegida**
Multiplataforma (Flutter)	Buen rendimiento e interfaz uniforme.	Requiere aprender Dart; no reutiliza el conocimiento de React.	Descartada
PWA (web instalable)	Sin tiendas; reutiliza la web.	Soporte limitado de notificaciones y de integración con el dispositivo, sobre todo en iOS.	Descartada

1.4 Arquitectura de la Información

Se mantiene el criterio de profundidad progresiva y mínimo privilegio del plan web, adaptado al móvil: sección pública → inicio de sesión → home por rol con navegación inferior (bottom tabs) de hasta 5 destinos. La autorización real siempre se resuelve en el backend (JWT + RBAC); la app solo oculta lo que el rol no puede usar.



Todos los roles acceden también a la sección pública. El Administrador utiliza únicamente el sistema web.

Figura 1. Mapa de navegación de la app por rol — "Educar para Transformar" (App Móvil).

Matriz de acceso por rol (aplicación móvil):

Sección / Pantalla	Público	Estudiante	Padre	Docente
Información institucional / noticias / galería	✔	✔	✔	✔
Dejar opinión	✔	✔	✔	✔
Inicio de sesión / biometría	—	✔	✔	✔
Datos académicos propios	—	✔	✔ hijos	—
Inscripción deportes / transporte / comedor	—	—	✔	—
Cursos y materias a cargo	—	—	—	✔
Notificaciones push	—	✔	✔	✔
Perfil y cierre de sesión	—	✔	✔	✔

El rol Administrador (ABM, reportes, usuarios y permisos) opera únicamente desde el sistema web.

1.5 Arquitectura de la aplicación de software

Aplicación de los principios

- **Separación de responsabilidades:** dentro de la app, pantallas, lógica de presentación y acceso a datos en capas independientes.
- **Fuente única de verdad:** las reglas de negocio (máximo 2 deportes, choque de horarios, duplicados) viven solo en la API; la app muestra los rechazos que esta devuelve. Puede validar en el cliente únicamente por usabilidad.
- **Mínimo privilegio:** autenticación centralizada (JWT + RBAC); cada rol ve solo su navegación.
- **Bajo acoplamiento y alta cohesión:** módulos independientes (Acceso, Institucional, Alumnos, Docentes, Notificaciones).
- **Seguridad en el dispositivo:** token en almacenamiento seguro, expiración corta, cierre de sesión que revoca el token y el registro de push.

Componentes

- App Móvil (React Native + Expo) para Android e iOS — **nueva**.
- Sitio Web, API Gateway / Autenticación (JWT-RBAC), Servicios Alumnos, Profesores y Administración/Reportes — **existentes**.
- Servicio de notificaciones push (FCM / APNs) y módulo de registro de dispositivos en el backend — **nuevos**.
- Base de datos relacional centralizada — existente, con una tabla nueva de dispositivos.

Componentes funcionales de la app

- **Acceso:** login, biometría, manejo y renovación de sesión.
- **Institucional:** noticias, galería y opiniones.
- **Alumnos:** datos académicos e inscripciones a deportes, transporte y comedor.
- **Docentes:** consulta de materias y cursos asignados.
- **Notificaciones:** permisos, registro del dispositivo y recepción.
- **Núcleo:** cliente HTTP, almacenamiento seguro, caché y manejo de errores.

Restricciones

- Máximo 2 deportes por alumno sin superposición horaria (RF-08); una única inscripción activa al transporte (RF-09); sin inscripciones duplicadas (RF-15). Todas se validan en el backend.

- Cumplimiento de la Ley 25.326: sin datos sensibles del menor en el texto de las notificaciones ni en la pantalla bloqueada.
- La app solo se comunica por HTTPS y nunca guarda el token en almacenamiento sin cifrar.
- Cumplimiento de las políticas de Google Play y App Store (política de privacidad, declaración de datos, clasificación por edad).

Conectores

- HTTPS / JSON mediante API REST versionada (/api/v1) entre la app y el API Gateway.
- FCM (Android) y APNs (iOS) para notificaciones push, disparadas por el backend.
- Keystore / Keychain (vía almacenamiento seguro del framework) entre la app y el dispositivo.

Tipo de Arquitectura de Software

Arquitectura Cliente-Servidor multicapas: la app es un **cliente liviano** del backend existente (monolito modular con API REST). Internamente sigue el patrón **MVVM por módulos** (pantalla → hook/ViewModel → repositorio). No se modifica la lógica de negocio del servidor, cumpliendo RNF-06.

Impacto sobre el backend existente

Ajustes a confirmar según el estado real de la API del plan web:

Cambio	Descripción	Requerimiento
Registro de dispositivos	Endpoint POST /api/v1/dispositivos para asociar el token push al usuario, y baja al cerrar sesión.	RFM-10
Renovación de sesión	Endpoint POST /api/v1/auth/refresh (refresh token) si aún no existe.	RFM-11
Perfil del usuario	Endpoint GET /api/v1/me con rol y, para el padre, la lista de hijos.	RFM-01, RFM-04
Disparo de notificaciones	El backend emite el push al confirmarse una inscripción o publicarse una novedad.	RFM-10
Versionado de API	Prefijo /api/v1 y versión mínima soportada de la app para evitar romper clientes instalados.	RNFM-08
Auditoría	Registrar el origen “móvil” en las operaciones críticas.	RNFM-11

Tecnologías (propuesta)

Aspecto	Tecnología / herramienta
Gestión de Proyecto	Jira (tablero Kanban de la app, por equipo y por integrante) – bitácora de evolución
Framework móvil	React Native + Expo (TypeScript) – Android e iOS con una base de código
Navegación	React Navigation / Expo Router
Estado y datos remotos	TanStack Query (caché de consultas) + Context o Zustand para la sesión
Cliente HTTP	Axios o fetch, con interceptores para el JWT y para errores de negocio
Almacenamiento seguro	expo-secure-store (Keystore / Keychain); AsyncStorage solo para caché no sensible
Biometría	expo-local-authentication
Notificaciones	expo-notifications + Firebase Cloud Messaging (Android) y APNs (iOS)
Maquetación	Figma – wireframe, mockup y prototipo navegable
Repositorio de Software	Git + GitHub (repositorio propio de la app, rama por historia de usuario y pull requests)

Aspecto	Tecnología / herramienta
Pruebas	Jest + React Native Testing Library; Postman para la API; pruebas manuales en dispositivos y emuladores
Build y distribución	EAS Build; Google Play Console (prueba interna) y TestFlight (iOS, si se dispone de cuenta)
Backend	API REST existente del plan web, con los ajustes indicados en el punto anterior

Gestión de Proyecto (Kanban)

Se utilizará Jira con un tablero Kanban propio de la app para registrar la evolución del trabajo. Para la presentación final se mostrará la bitácora del equipo y la de cada integrante, con las columnas To Do → In Progress → Done.

Maquetación

- **Wireframe:** bocetos de baja fidelidad de login, home por rol, mis hijos, inscripción a deportes, transporte/comedor, notificaciones y perfil.
- **Mockup:** diseño de media/alta fidelidad con paleta, tipografía y componentes según Material Design 3 / HIG, incluyendo estados de carga, vacío, error y sin conexión.
- **Prototipo:** prototipo navegable en Figma con los flujos clave (inicio de sesión, inscripción a un deporte con rechazo por superposición, recepción de una notificación).

Patrones de Diseño

- **MVVM:** separa la pantalla (vista) de su lógica de presentación (hooks / ViewModel).
- **Repository:** acceso a la API y a la caché desacoplado de la lógica de presentación.
- **Adapter:** convierte los datos que devuelve la API en los modelos que usa la app.
- **Singleton:** una única instancia del cliente HTTP y de la configuración de sesión.
- **Observer:** actualización reactiva de la interfaz ante cambios de estado y eventos de notificación.

Gráfico de la arquitectura de software

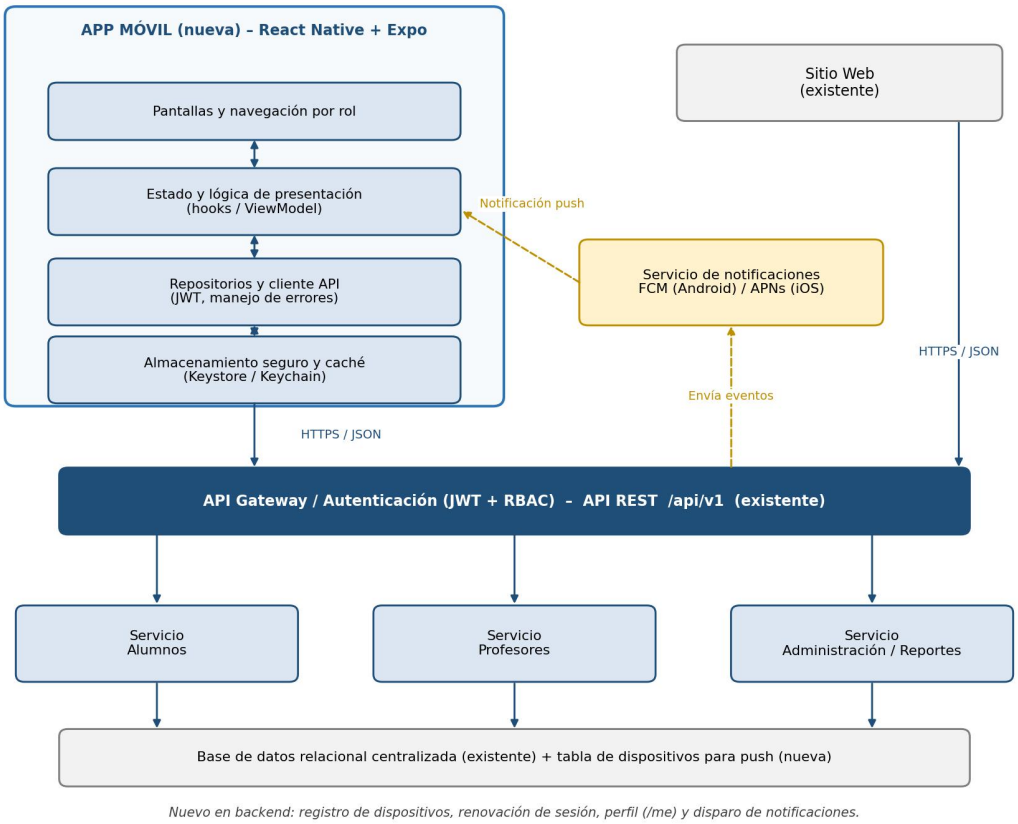


Figura 2. Arquitectura de software con la App Móvil integrada — "Educar para Transformar".

Diagrama de componentes (código PlantUML de referencia):

```
@startuml
skinparam componentStyle rectangle

package "App Móvil (nueva)" {
  [Pantallas y navegación] as UI
  [Estado y lógica (hooks / ViewModel)] as VM
  [Repositorios y cliente API] as Repo
  [Almacenamiento seguro y caché] as Store
}

package "Capa de Presentación existente" {
  [Sitio Web] as Web
}

package "Capa de Lógica de Negocio (existente)" {
  [API Gateway / Auth (JWT-RBAC)] as Auth
  [Servicio Alumnos] as SvcAlumnos
  [Servicio Profesores] as SvcProfesores
  [Servicio Administración/Reportes] as SvcAdmin
  [Servicio de Notificaciones (nuevo)] as SvcNotif
}

cloud "FCM / APNs" as Push
database "Base de Datos" as DB

UI --> VM
VM --> Repo
Repo --> Store
Repo --> Auth : HTTPS/JSON
Web --> Auth : HTTPS/JSON
Auth --> SvcAlumnos
Auth --> SvcProfesores
```

```
Auth --> SvcAdmin
Auth --> SvcNotif
SvcNotif --> Push : envía push
Push ..> UI : notificación
SvcAlumnos --> DB
SvcProfesores --> DB
SvcAdmin --> DB
SvcNotif --> DB
@enduml
```

2. OBJETIVOS DEL PROYECTO (formato SMART)

Objetivo general: Desarrollar y poner a disposición, en versión de prueba interna en las tiendas, la aplicación móvil del Centro Educativo “Educar para Transformar” como nuevo canal del Sistema de Gestión web para padres, estudiantes y docentes, antes del inicio de operaciones en marzo de 2027.

Objetivos específicos (SMART):

Código	Objetivo SMART
OBJ-M01	Implementar el inicio de sesión seguro (JWT en almacenamiento seguro) y la navegación por rol para los 3 perfiles móviles (Estudiante, Padre, Docente) sobre la API existente, al finalizar el Sprint 1.
OBJ-M02	Implementar la consulta de datos académicos (padre y estudiante) y de asignaciones (docente), con tiempo de respuesta ≤ 3 s y consulta sin conexión de la última sincronización, al finalizar el Sprint 2.
OBJ-M03	Implementar la inscripción a deportes, transporte y comedor cumpliendo el 100% de las reglas de negocio (máximo 2 deportes, sin superposición, sin duplicados) según las respuestas de la API, al finalizar el Sprint 3.
OBJ-M04	Implementar las notificaciones push, con recepción en ≤ 1 minuto en al menos el 95% de las pruebas, y el acceso público institucional con opiniones, al finalizar el Sprint 4.
OBJ-M05	Entregar el prototipo navegable en Figma (wireframe, mockup y prototipo) con los flujos clave antes de finalizar el Sprint 2 (semana 4).
OBJ-M06	Generar el build de release firmado, publicarlo en prueba interna (Google Play y, si se dispone de cuenta, TestFlight) y entregar la bitácora Kanban de equipo e individual en Jira antes de finalizar el Sprint 5 (semana 10).

3. CRONOGRAMA DE ACTIVIDADES – PARTE APLICACIÓN MÓVIL

3.1 Descripción de las actividades

ETAPA	TAREAS	DURACIÓN (HS)	RESULTADOS ESPERADOS	ALUMNO RESP.
Planificación del Proyecto	Ambos	8	Alcance móvil definido, backlog inicial, plan de sprints, tablero Kanban y repositorio configurados.	Trangoni / Mazza
Estudio de Requerimientos	Análisis	10	RFM y RNFM validados, historias de usuario y casos de uso móviles, matriz de acceso móvil.	Trangoni / Mazza
Modelado y Arquitectura	Análisis / API	10	Arquitectura de la app, diagramas de componentes y de secuencia (login, inscripción, push) y contrato de API (endpoints existentes y nuevos).	Mazza
Diseño UX/UI	Diseño / UX	20	Mapa de navegación, wireframes, mockups y prototipo navegable en Figma.	Trangoni
Codificación	Desarrollo	72	Módulos Acceso, Institucional, Alumnos, Docentes y Notificaciones; navegación por rol y manejo de sesión.	Trangoni / Mazza
Integración con API y notificaciones push	Desarrollo / API	16	Consumo de los endpoints reales, manejo de errores de negocio, registro de dispositivos y recepción de push.	Mazza
Pruebas en dispositivos	QA	24	Casos de prueba ejecutados en dispositivos y emuladores Android/iOS; pruebas de reglas de negocio, seguridad básica y rendimiento (≤ 3 s).	Trangoni / Mazza

ETAPA	TAREAS	DURACIÓN (HS)	RESULTADOS ESPERADOS	ALUMNO RESP.
Implementación o Publicación	DevOps	14	Builds de release firmados, publicación en prueba interna, política de privacidad, ficha de tienda, documentación y bitácora Kanban final.	Trangoni / Mazza

Duración total estimada: 174 horas de trabajo del equipo (aprox. 84 h Trangoni y 90 h Mazza).

3.2 Gráfico del Diagrama de Gantt

Planificación estimada en 10 semanas. Las celdas coloreadas indican la/s semana/s en que se ejecuta cada etapa.

Etapa	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Planificación	■									
Est. Requerimientos	■	■								
Modelado / Arquitectura		■								
Diseño UX/UI		■	■							
Codificación		■	■	■	■	■	■	■		
Integración API y push				■	■	■	■	■		
Pruebas en dispositivos								■	■	
Publicación									■	■

3.3 Diagrama de PERT

Secuencia de dependencias entre etapas con la duración estimada (en horas). La ruta crítica atraviesa todas las etapas en el orden indicado.

[A] Planif. (8h) → [B] Requerim. (10h) → [C] Modelado (10h) → [D] Diseño (20h) → [E] Codificación (72h) → [F] Integración API/push (16h) → [G] Pruebas (24h) → [H] Publicación (14h) | Ruta crítica = 174 h

Nota: en el Gantt algunas etapas se solapan porque los dos integrantes trabajan en paralelo; la ruta crítica suma las horas de esfuerzo. Se recomienda construir el PERT definitivo en Jira una vez confirmadas las estimaciones por historia.

4. BACKLOG DEL SPRINT

Se presenta un backlog por cada historia de usuario móvil (Integrantes 1 y 2). Los requerimientos RFM-11 y RFM-12 se abordan como tareas transversales dentro de HU-M01 y HU-M05.

Integrante 1 – Trangoni, Fabián

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M01	Iniciar sesión en la app y mantener la sesión de forma segura (padre, estudiante, docente) – RFM-01, RFM-11	Alta	To Do	1. Diseñar la pantalla de login (correo y contraseña). 2. Consumir el endpoint de autenticación y guardar el token en almacenamiento seguro. 3. Redirigir al home según el rol. 4. Manejar expiración/renovación y cierre de sesión. 5. Opcional: desbloqueo biométrico.	- Credenciales inválidas muestran un mensaje que no revela qué dato falló. - El token no se guarda en almacenamiento sin cifrar. - Cada rol ve solo su navegación. - Al expirar la sesión se solicita un nuevo ingreso.

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M02	Inscribir a mi hijo a deportes desde la app (máx. 2, sin choque de horario) – RFM-06	Alta	To Do	1. Listar deportes con horario y profesor (API). 2. Elegir al hijo si hay más de uno. 3. Enviar la inscripción y mostrar los rechazos de la API. 4. Confirmar y reflejar la inscripción en el perfil del alumno.	- No permite un 3.er deporte (mensaje claro). - Informa los horarios superpuestos rechazados por la API. - La inscripción se ve luego en la app y en la web. - Sin conexión, informa que no puede inscribir.

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M03	Inscribir a mi hijo al servicio de transporte y al comedor – RFM-07, RFM-08	Alta	To Do	1. Mostrar los 4 recorridos y sus paradas. 2. Detectar inscripción activa y ofrecer modificarla. 3. Registrar recorrido y comedor. 4. Mostrar la confirmación.	- Permite elegir 1 de 4 recorridos. - Impide una 2.ª inscripción activa al transporte. - Evita inscripción duplicada al comedor. - La confirmación queda visible en el perfil.

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M07	Consultar información institucional y dejar una opinión sin iniciar sesión – RFM-02, RFM-03	Media	To Do	1. Pantallas de institucional, noticias y galería. 2. Formulario de opinión (nombre, comentario, calificación). 3. Validar campos obligatorios. 4. Enviar y mostrar confirmación.	- No solicita usuario ni contraseña. - Rechaza el envío con campos obligatorios vacíos. - La opinión queda publicada o en moderación según la configuración.

Integrante 2 – Mazza, Bruno

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M04	Recibir notificaciones push de inscripciones y novedades – RFM-10	Alta	To Do	1. Solicitar el permiso de notificaciones con explicación previa. 2. Obtener el token del dispositivo y registrarlo en el backend. 3. Disparar el push al confirmarse una inscripción o novedad. 4. Abrir la pantalla correspondiente al tocar la notificación. 5. Eliminar el token al cerrar sesión.	- La notificación llega en ≤ 1 minuto en las pruebas. - El texto no incluye datos sensibles del menor. - Tras cerrar sesión no se reciben notificaciones del usuario anterior.

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M05	Consultar los datos académicos de mi hijo (padre) o los propios (estudiante) – RFM-04, RFM-05, RFM-12	Alta	To Do	1. Obtener el perfil y la lista de hijos (GET /me). 2. Mostrar nivel, curso, materias, horarios e inscripciones. 3. Guardar en caché la última consulta para uso sin conexión. 4. Restringir la consulta a los hijos propios.	- El padre ve solo a sus hijos. - El estudiante ve solo sus datos. - Sin red, muestra la última información con su fecha de sincronización. - La carga tarda ≤ 3 s.

ID	Título / Historia de Usuario	Prioridad	Estado	Tareas	Criterios de Aceptación
HU-M06	Consultar las materias y cursos que tengo asignados (docente) – RFM-09	Media	To Do	1. Obtener las asignaciones del docente. 2. Listar materia, curso y nivel. 3. Mostrar el detalle por curso. 4. Resolver los estados vacío y de error.	- Muestra solo las asignaciones propias. - Refleja los cambios hechos desde la web. - Informa “sin asignaciones” si no hay datos.

5. PLAN DE SPRINT

Plan organizado por semana para la etapa de Programación, indicando los requerimientos (historias de usuario) incluidos en cada sprint. Se estiman sprints de 2 semanas.

Sprint	Período	Requerimientos incluidos	Responsable
Sprint 1	Semanas 1–2	Configuración del entorno (Expo, repositorio, cliente HTTP, almacenamiento seguro) · HU-M01 (RFM-01, RFM-11) Inicio de sesión y sesión segura · Navegación base por rol.	Trangoni (HU-M01) / Mazza (base técnica)
Sprint 2	Semanas 3–4	HU-M05 (RFM-04, RFM-05, RFM-12) Datos académicos y caché · HU-M06 (RFM-09) Materias y cursos del docente · Prototipo navegable en Figma.	Mazza / Trangoni (prototipo)
Sprint 3	Semanas 5–6	HU-M02 (RFM-06) Inscripción a deportes · HU-M03 (RFM-07, RFM-08) Transporte y comedor.	Trangoni
Sprint 4	Semanas 7–8	HU-M04 (RFM-10) Notificaciones push y endpoint de dispositivos · HU-M07 (RFM-02, RFM-03) Institucional y opiniones.	Mazza (HU-M04) / Trangoni (HU-M07)
Sprint 5	Semanas 9–10	Pruebas integrales en dispositivos · Corrección de defectos · Build de release y publicación en prueba interna · Documentación y bitácora Kanban final.	Trangoni / Mazza

6. GESTIÓN DE RIESGOS

ID	Riesgo	Prob.	Impacto	Mitigación
R-01	La API web cambia o presenta defectos durante el desarrollo de la app.	Media	Alto	API versionada (/api/v1), contrato acordado y probado con Postman; datos simulados (mocks) para avanzar en paralelo.
R-02	Demora o rechazo en la revisión de Google Play / App Store.	Media	Alto	Preparar política de privacidad y declaración de datos desde el Sprint 4; usar prueba interna / TestFlight; enviar builds con anticipación.
R-03	Costo o trámite de las cuentas de desarrollador (sobre todo para iOS).	Media	Medio	Priorizar Android; confirmar con la cátedra el alcance de la entrega en iOS durante el Sprint 1.
R-04	Fragmentación de dispositivos y versiones de Android.	Alta	Medio	Matriz mínima de dispositivos y emuladores; usar librerías estándar de Expo; probar en al menos 3 versiones.
R-05	Exposición de datos de menores (dispositivo perdido, notificaciones en pantalla bloqueada).	Baja	Alto	Token en almacenamiento seguro, expiración corta, biometría, notificaciones sin datos sensibles y revocación del token al cerrar sesión.
R-06	Notificaciones push no entregadas (permisos denegados, ahorro de batería).	Media	Medio	Explicar el permiso antes de pedirlo; toda la información debe estar disponible al abrir la app; no depender del push para procesos críticos.
R-07	Curva de aprendizaje de React Native / Expo.	Media	Medio	Spike técnico en el Sprint 1; aprovechar el conocimiento previo de React y la documentación oficial.
R-08	Equipo reducido (2 integrantes) y carga desigual de horas.	Media	Medio	Backlog priorizado, revisión semanal del tablero Kanban y alcance mínimo viable: login + consultas + inscripciones.

7. ESTRATEGIA DE PRUEBAS Y PUBLICACIÓN

7.1 Pruebas

Tipo de prueba	Alcance	Herramienta
Unitarias	Validaciones, utilidades, hooks y repositorios.	Jest
Componentes / UI	Pantallas principales y estados de carga, vacío, error y sin conexión.	React Native Testing Library
Integración con la API	Cada endpoint usado, incluidos los rechazos de reglas de negocio.	Postman, ambiente de pruebas
Funcionales por historia	Verificación de los criterios de aceptación de HU-M01 a HU-M07.	Casos de prueba manuales
En dispositivos	Al menos 3 versiones de Android (emulador o físico) y 1 iOS; pantallas chica y grande.	Dispositivos y emuladores
Seguridad básica	HTTPS, token en almacenamiento seguro, sin datos sensibles en logs ni notificaciones.	Revisión de código y checklist
Rendimiento	Arranque y consultas ≤ 3 s en red 4G.	Medición manual / perfilador
Usabilidad	Prueba con al menos 3 personas sin conocimientos técnicos.	Observación y encuesta breve

7.2 Publicación y mantenimiento

- Ícono, pantalla de inicio (splash), número de versión y de build, y firma de la app (keystore).
- Política de privacidad alineada con la Ley 25.326, ficha de la tienda (descripción y capturas), clasificación por edad y declaración de seguridad de datos.
- Publicación en prueba interna (Google Play) y TestFlight (iOS, si se dispone de cuenta) antes de una eventual publicación abierta.
- Versionado semántico, versión mínima soportada de la app y estrategia de actualización de la API sin romper clientes instalados.
- Monitoreo de errores en producción (por ejemplo, Firebase Crashlytics o Sentry) y canal de reporte para los usuarios.

Fin del Plan de Trabajo – Parte Aplicación Móvil.